The Last Mile of an API Integration Is a Decision

An independent Olostep case study in partial results, recovery, and executable documentation

Amy V · Technical case study for Hamza and the Olostep team

A batch can finish successfully while the application still lacks the information it needs. That distinction is easy to describe and surprisingly easy to erase in code. A green status badge, a completed polling loop, or a convenient example can encourage a developer to treat the end of an operation as proof of a complete result.

For a web-data pipeline, those are separate claims. The service may have finished processing every submitted URL while one page failed, another returned no usable content, and a third remains unavailable to the consumer. The application needs to understand those outcomes before using the dataset downstream.

I built the **Olostep Integration Reliability Lab** around that boundary. It is a small, independent reference implementation of one developer journey: submit a batch, follow its progress, enumerate its outcomes, retrieve content, and export evidence. The intent is to make operational decisions visible and teachable.

The current verification is local and fixture-based. The optional REST adapter is implemented but has not been exercised against a live Olostep account. That limit is part of the case study, not something the visual presentation should obscure.

Start where the developer's decision begins

Olostep's documentation covers a broad set of capabilities, including scraping, batching, crawling, discovery, and research. A useful reference does not need to reproduce that whole surface. It needs to choose a situation precisely enough that the reader understands why one path is appropriate.

This sample begins with known URLs. That makes batching a coherent choice and removes the separate uncertainty of discovering relevant sources. Search can be added later, but it introduces a different question: whether the returned pages were the right pages in the first place. Mixing that concern into the initial example would make an extraction failure harder to explain.

The batch guide provides the essential lifecycle. The sample adds an explicit consumer policy around it: how long to wait, what to retry, when to stop, and what evidence to preserve. These are application decisions, not assertions about how Olostep ought to implement its service. Batch workflow

Three outcomes are more useful than one success flag

The workflow exports three application-level outcomes:

- **Complete:** enumeration finished and the retrieved content is usable under this sample's narrow check.
- **Partial:** enumeration finished, but at least one source failed or has unresolved content.
- **Incomplete:** the client could not finish determining the result.

“Usable” has a deliberately modest meaning here: a nonempty Markdown string. It does not establish factual correctness, relevance, completeness of a document, or suitability for a particular model. Those require additional evaluation.

In the partial-result fixture, three sources are expected. Two yield content and one appears in the failed inventory. The exported result preserves that failure. It does not silently omit the page and present the remaining two as the whole dataset. The runner checks both the original item identities and the provider total when available. Equal counts do not prove that the right three items returned. A resumed run without the original manifest explicitly reports its weaker, provider-count-only verification; missing count data leaves completeness unknown.

This is a documentation lesson as much as a software lesson: explain the meaning of success before teaching the reader how to print it.

A retry is a decision about uncertainty

It is tempting to put every request inside the same retry helper. That produces short code, but it treats materially different operations as equivalent.

The critical distinction in this sample is between creating work and observing work. If a creation request times out, the client may not know whether the server accepted it. Repeating that request can create another operation. A local request identifier does not solve this unless the provider offers and guarantees matching idempotency semantics for that endpoint.

Consequently, the sample makes one creation attempt. An ambiguous network or server failure produces an explicit unknown outcome and instructions to reconcile before resubmitting. Once a batch ID is available, subsequent commands can resume reads against the existing operation. The CLI saves a minimal checkpoint before polling: batch ID, execution mode, and expected item identities. Atomic replacement protects readers from a half-written file. It cannot eliminate the gap between provider acceptance and local persistence; that case still requires reconciliation.

Reads have a separate, bounded policy: three attempts for selected transient failures, short exponential delays, and a shared deadline. Authentication failures do not enter that retry loop. A credential rejected once is unlikely to become valid because the program repeated itself.

Reads honor Retry-After in seconds or HTTP-date form, with small jitter in live mode. A delay exceeding the remaining budget stops the run instead of triggering an early retry. Tests inject time and jitter to verify this policy.

Pagination is part of correctness

Retrieving the first page of a result set is not the same as retrieving the result set. The runner follows the documented cursor pattern, stops on an absent terminal cursor, and rejects repeated cursors. It also caps the number of pages and rejects duplicate item identities rather than counting them twice.

These checks turn a potentially silent reporting error into a visible integration failure. The response is less cosmetically smooth, but more useful to a developer trying to understand what happened. The underlying model follows the separation between item listing and stored-content retrieval in the public reference. Batch items, Retrieve content

A webhook should invite reconciliation

The accompanying SQLite inbox demonstrates a second boundary: receiving a notification is different from proving the underlying result.

It stores event IDs with a digest of the semantic payload. Receiving the same event again produces a duplicate decision. Reusing an ID with changed data produces a conflict. Because the record is persistent, duplicate detection survives a restart. Delivery-attempt metadata is excluded from the digest so an ordinary redelivery does not look like a payload conflict.

This is deliberately an isolated simulator, not a production receiver. It proves the inbox's duplicate-handling behavior; it does not prove end-to-end exactly-once processing. A production system would need authentication, durable work scheduling, crash reconciliation and a clear transaction boundary around side effects.

Olostep's webhook documentation describes completion notifications and delivery retries. The application should use such notifications to initiate authenticated status retrieval, rather than trusting an incoming payload as the complete source of truth. Webhook documentation

Evidence should travel with the result

The JSON export records item outcomes, retry decisions, the batch identifier, execution mode, policy and limitations. A Markdown export presents the same core evidence for human review.

Valid usage returned by batch information is labeled provider-reported in live mode or fixture-derived in simulation. Missing or invalid usage stays unknown; a genuine zero remains zero. Neither item counts nor synthetic usage are billing measurements, and a provider field is not an independently reconciled invoice. Local timing is not a provider benchmark. The browser demonstration uses virtual time so a reviewer can explore an expired deadline immediately; that time is explicitly labeled as simulated. The CLI measures elapsed local execution instead.

Those distinctions prevent a polished interface from lending more authority to a number than the evidence supports. A developer should not need to inspect the source code to discover that a displayed result was simulated.

Test the explanation, not only the implementation

The offline suite exercises the claims made in the guide: partial completion, bounded retries, retained resume identifiers, non-retried ambiguous creation, cursor loops, missing content, unknown credits, and durable duplicate detection. Regression cases also reject empty resumed results and wrong identities with matching counts, and verify checkpoint recovery and deadline-aware Retry-After. Distinct fixture documents include code, a table and an authentication guide; they exercise content handling, not the quality of real web extraction.

The fixtures make those conditions reproducible without credentials or network variability. That is valuable for learning and regression testing. It is not a substitute for a small live compatibility run. The next verification step should use permitted public URLs, a limited credit budget, and preserved sanitized response shapes to check the adapter against the current service.

After compatibility is established, a separate usability exercise could ask a developer unfamiliar with the project to complete the quickstart and explain a partial result. Measure where they hesitate and whether the recovery instructions resolve the problem. Until then, claims about faster onboarding or reduced support work would be hypotheses.

What this could contribute

This reference offers three connected artifacts: working code a developer can inspect, documentation they can execute, and a short demonstration they can follow. Its value is their agreement. The article describes decisions that exist in the runner; the tests exercise those decisions; the console reveals their evidence.

That is the contribution I would want to bring to API documentation: help users understand the system well enough to make good decisions when the first successful request is no longer the whole problem.

Independent sample. No affiliation, endorsement, production adoption, or live performance claim is implied. Public-source references were reviewed on October 14, 2026. The accompanying verification record states the checks actually run.